Федеральное государственное автономное образовательное учреждение
высшего образования
Национальный исследовательский университет ИТМО
Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия

Дисциплина «Распределённые системы хранения данных»

Отчёт
по лабораторной работе №1

Вариант: 368748

Студент:
Чусовлянов Максим Сергеевич
Садовой Григорий Владимирович
Группа P3307

Преподаватель:
Максимов Андрей Николаевич

г. Санкт-Петербург, 2026 г.

Оглавление

Задание ... 3
Выполнение ... 4
Этап 1. Инициализация кластера БД ... 4
Этап 2. Конфигурация и запуск сервера БД ... 4
Конфигурация сервера через ALTER SYSTEM ... 4
Обоснование параметров OLAP ... 5
Параметры логирования ... 8
Итоговый postgresql.auto.conf ... 8
Этап 3. Дополнительные табличные пространства и наполнение базы ... 9
Создание директорий и табличных пространств ... 9
Создание базы данных ... 9
Создание роли и раздача прав ... 9
Наполнение баз от имени новой роли ... 10
Список табличных пространств и объектов ... 10
Демонстрация ... 13
Инициализация кластера ... 13
Запуск сервера и проверка порта ... 14
Содержимое pg_hba.conf ... 14
Применение конфигурации (postgresql.auto.conf) ... 15
Создание табличных пространств и базы ... 15
Наполнение баз от имени lab1_user ... 16
Список табличных пространств и объектов ... 17
Исходный код ... 18
Вывод ... 19

Задание

Лабораторная работа №1

Введите вариант: 36874

Внимание! У разных вариантов разный текст задания!

Цель работы - на выделенном узле создать и сконфигурировать новый кластер БД Postgres, саму БД, табличные пространства и новую роль, а также произвести наполнение базы в соответствии с заданием. Отчёт по работе должен содержать все команды по настройке, скрипты, а также измененные строки конфигурационных файлов.

Способ подключения к узлу из сети Интернет через helios:
`ssh -J sXXXXXX@helios.cs.ifmo.ru:2222 postgresY@pgZZZ`
Способ подключения к узлу из сети факультета:
`ssh postgresY@pgZZZ`
Номер выделенного узла `pgZZZ`, а также логин и пароль для подключения Вам выдаст преподаватель.

Этап 1. Инициализация кластера БД

- Директория кластера: `$HOME/aoj42`
- Кодировка: UTF8
- Локаль: английская
- Параметры инициализации задать через переменные окружения

Этап 2. Конфигурация и запуск сервера БД

- Способы подключения: 1) Unix-domain сокет в режиме peer; 2) сокет TCP/IP, только localhost
- Номер порта: `9748`
- Способ аутентификации TCP/IP клиентов: по паролю MD5
- Остальные способы подключений запретить.
- Настроить следующие параметры сервера БД:
  - max_connections
  - shared_buffers
  - temp_buffers
  - work_mem
  - checkpoint_timeout
  - effective_cache_size
  - fsync
  - commit_delay

  Параметры должны быть подобраны в соответствии со сценарием OLAP:
  9 одновременных пользователей, пакетная запись/чтение данных по 256МБ.
- Директория WAL файлов: `$PGDATA/pg_wal`
- Формат лог-файлов: `.csv`
- Уровень сообщений лога: `WARNING`
- Дополнительно логировать: попытки подключения и завершение сессий

Этап 3. Дополнительные табличные пространства и наполнение базы

- Создать новые табличные пространства для временных объектов: `$HOME/ydp10, $HOME/zcc31`
- На основе template1 создать новую базу: `bestbluemath`
- Создать новую роль, предоставить необходимые права, разрешить подключение к базе.
- От имени новой роли (не администратора) произвести наполнение ВСЕХ созданных баз тестовыми наборами данных. ВСЕ табличные пространства должны использоваться по назначению.
- Вывести список всех табличных пространств кластера и содержащиеся в них объекты.

Выполнение

Этап 1. Инициализация кластера БД

Устанавливаем переменные окружения и инициализируем кластер:

```
export PGDATA=$HOME/aoj42
export PGLOCALE=en_US.UTF-8
export PGENCODE=UTF8
export LANG=en_US.UTF-8
export LC_ALL=en_US.UTF-8

initdb -D "$PGDATA" \
  --encoding=$PGENCODE \
  --locale=$PGLOCALE \
  --lc-messages=$PGLOCALE \
  --lc-monetary=$PGLOCALE \
  --lc-numeric=$PGLOCALE \
  --lc-time=$PGLOCALE \
  --username=postgres0
```

WAL-директория `$PGDATA/pg_wal` создаётся автоматически при инициализации (расположение по умолчанию совпадает с требованием задания).

Сразу после `initdb` заменяем `pg_hba.conf`, чтобы настроить нужные методы подключения до первого запуска:

```
cat > "$PGDATA/pg_hba.conf" << 'EOF'
# TYPE  DATABASE  USER  ADDRESS        METHOD
# Unix-domain socket - peer authentication
local   all       all                  peer
# TCP/IP - localhost only - MD5
host    all       all   127.0.0.1/32   md5
host    all       all   ::1/128        md5
EOF
```

Запускаем сервер (первоначально на порту по умолчанию):

```
pg_ctl -D "$PGDATA" -l "$PGDATA/server.log" start
```

Этап 2. Конфигурация и запуск сервера БД

Конфигурация сервера через ALTER SYSTEM

Подключаемся к серверу и применяем параметры (записываются в `postgresql.auto.conf`):

```
-- Принимать подключения только с localhost
ALTER SYSTEM SET listen_addresses = 'localhost';
-- Порт по варианту
ALTER SYSTEM SET port = 9748;

-- OLAP: 9 пользователей, пакетные операции 256 МБ
ALTER SYSTEM SET max_connections = 20;
```

```
ALTER SYSTEM SET superuser_reserved_connections = 3;
ALTER SYSTEM SET shared_buffers = '512MB';
ALTER SYSTEM SET temp_buffers = '256MB';
ALTER SYSTEM SET work_mem = '256MB';
ALTER SYSTEM SET effective_cache_size = '4GB';
ALTER SYSTEM SET maintenance_work_mem = '256MB';

ALTER SYSTEM SET checkpoint_timeout = '1h';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
ALTER SYSTEM SET fsync = on;
ALTER SYSTEM SET commit_delay = 0;
ALTER SYSTEM SET commit_siblings = 5;

ALTER SYSTEM SET default_statistics_target = 500;
ALTER SYSTEM SET random_page_cost = 4.0;
ALTER SYSTEM SET effective_io_concurrency = 2;
ALTER SYSTEM SET max_worker_processes = 4;
ALTER SYSTEM SET max_parallel_workers_per_gather = 2;
ALTER SYSTEM SET max_parallel_workers = 4;

ALTER SYSTEM SET log_destination = 'csvlog';
ALTER SYSTEM SET logging_collector = on;
ALTER SYSTEM SET log_filename = 'postgresql-%Y-%m-%d_%H%M%S.csv';
ALTER SYSTEM SET log_min_messages = 'warning';
ALTER SYSTEM SET log_connections = on;      -- попытки подключения
ALTER SYSTEM SET log_disconnections = on;   -- завершение сессий
ALTER SYSTEM SET log_checkpoints = on;
ALTER SYSTEM SET log_lock_waits = on;
```

Перезапускаем сервер для применения параметров, требующих рестарта:

```
pg_ctl -D $PGDATA restart
```

Обоснование параметров OLAP

Сценарий: 9 одновременных пользователей, пакетная запись/чтение по 256 МБ. Сервер: FreeBSD 14.4, 128 ГБ RAM, 16 CPU.

Параметр	Значение	Обоснование
max_connections	20	9 пользователей + 3 резерва для суперпользователя + запас. OLAP-системы не нуждаются в большом числе соединений: запросов мало, но они долгие.
shared_buffers	512MB	Буфер для кэширования страниц данных. Для OLAP желательно больше, но в условиях jail-окружения (ограничения fork) оптимальным оказалось 512 МБ. Кэширует наиболее горячие блоки аналитических таблиц.
temp_buffers	256MB	Буфер временных таблиц на сессию. Значение соответствует размеру пакета (256 МБ) по заданию:

Параметр	Значение	Обоснование
		промежуточные результаты аналитических запросов (агрегации, сортировки) помещаются в память без сброса на диск.
work_mem	256MB	Память на одну операцию сортировки или хэш-соединения. При 9 пользователях и сложных аналитических запросах суммарное потребление ≤ 9 × 256 МБ = 2,3 ГБ. Снижает число сортировок на диске, критичных для OLAP.
checkpoint_ timeout	1h	OLAP генерирует мало изменений по сравнению с OLTP. Редкие контрольные точки (раз в час) снижают накладные расходы и нагрузку на диск, что важно при длительных пакетных операциях.
effective_cache_ size	4GB	Подсказка планировщику об объёме доступного кэша ОС. Поощряет использование индексных сканов вместо последовательных при аналитических запросах по большим таблицам.
fsync	on	Гарантирует физическую запись данных на диск перед подтверждением транзакции. Несмотря на OLAP-характер нагрузки, целостность данных обязательна.
commit_delay	0	OLAP совершает единичные коммиты (в конце пакетных операций), а не тысячи транзакций в секунду. Группировка коммитов не даёт выигрыша — задержка не нужна.
superuser_ reserved_ connections	3	Резервирует 3 слота из max_connections исключительно для суперпользователя. Гарантирует административный доступ (диагностика, pg_cancel_backend) даже когда все 9 пользовательских слотов заняты долгими аналитическими запросами.
maintenance_work_ mem	256MB	Объём памяти для обслуживающих операций: VACUUM, ANALYZE, CREATE INDEX. OLAP-таблицы большие; 256 МБ позволяют строить индексы в памяти без промежуточной записи на диск, что сокращает время обслуживания.
checkpoint_ completion_target	0.9	Растягивает запись контрольной точки на 90% интервала checkpoint_timeout (т.е. 54 мин из 60). Равномерно распределяет I/O, избегая пиковой нагрузки при сбросе грязных блоков в конце интервала.

Параметр	Значение	Обоснование
commit_siblings	5	Дополнительное условие для commit_delay: задержка применяется только при наличии ≥5 активных транзакций. В OLAP-окружении с малым числом сессий это исключает лишнюю задержку при одиночных коммитах.
default_ statistics_target	500	Увеличивает гистограммы статистики с 100 до 500 значений. Для OLAP-запросов с фильтрацией по широким диапазонам (агрегации, GROUP BY) точная статистика критична: планировщик выбирает верный метод соединения и избегает sequential scan там, где нужен index scan.
random_page_cost	4.0	Стоимость случайного чтения страницы относительно последовательного (seq_page_cost = 1.0). Значение 4.0 соответствует HDD/SATA SSD. Поощряет планировщик выбирать sequential scan для больших аналитических выборок, а не index scan с дорогостоящими случайными переходами.
effective_io_ concurrency	2	Количество параллельных операций I/O, доступных одному запросу (bitmap heap scan). Значение 2 соответствует конфигурации с зеркалированием (RAID-1/2 диска); завышение привело бы к конкуренции за I/O в jail-окружении.
max_worker_ processes	4	Общий лимит фоновых рабочих процессов (параллельные запросы, autovacuum-воркеры). Значение 4 на 16-ядерном сервере оставляет ядра для обработки пользовательских сессий и не исчерпывает ресурсы в jail-окружении.
max_parallel_ workers	4	Сколько из max_worker_processes разрешено использовать для параллельных запросов суммарно. Равно max_worker_processes — весь пул доступен для распараллеливания аналитических запросов.
max_parallel_ workers_per_ gather	2	Максимум воркеров на один узел Gather в плане запроса. При 9 пользователях и 4 доступных воркерах значение 2 позволяет двум запросам выполняться параллельно одновременно ($2 \times 2 = 4$), не создавая очереди ожидания воркеров.

Параметры логирования

Параметр	Значение	Обоснование
log_destination	csvlog	CSV-формат структурирует записи лога для последующей загрузки в БД или инструменты анализа. В отличие от stderr, каждое событие — отдельная строка с чётко разделёнными полями.
logging_collector	on	Включает фоновый процесс сбора логов — необходимое условие для csvlog. Записи не теряются при высокой нагрузке, в отличие от прямой записи в stderr.
log_min_messages	warning	Уровень WARNING фиксирует нештатные ситуации (предупреждения, ошибки, фатальные события), не засоряя лог отладочными сообщениями уровня INFO/DEBUG. Подходит для продуктивного окружения.
log_connections	on	Логирует каждую попытку подключения. Требование задания. Позволяет отследить несанкционированные попытки доступа и диагностировать проблемы аутентификации.
log_disconnections	on	Логирует завершение каждой сессии с указанием её продолжительности. Требование задания. В сочетании с log_connections даёт полную картину жизненного цикла сессий.
log_checkpoints	on	Фиксирует каждую контрольную точку с информацией о записанных буферах и времени. Важно для мониторинга I/O-нагрузки при редких (раз в час) контрольных точках в OLAP-сценарии.
log_lock_waits	on	Записывает в лог ожидания блокировок, превышающие deadlock_timeout (по умолчанию 1 с). При параллельных пакетных загрузках данных позволяет выявить конкурентные конфликты между сессиями.

Итоговый postgresql.auto.conf

```
listen_addresses = 'localhost'
port = '9748'
max_connections = '20'
superuser_reserved_connections = '3'
shared_buffers = '512MB'
temp_buffers = '256MB'
work_mem = '256MB'
effective_cache_size = '4GB'
maintenance_work_mem = '256MB'
checkpoint_timeout = '1h'
checkpoint_completion_target = '0.9'
```

```
fsync = 'on'
commit_delay = '0'
commit_siblings = '5'
default_statistics_target = '500'
random_page_cost = '4.0'
effective_io_concurrency = '2'
max_worker_processes = '4'
max_parallel_workers_per_gather = '2'
max_parallel_workers = '4'
log_destination = 'csvlog'
logging_collector = 'on'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.csv'
log_min_messages = 'warning'
log_connections = 'on'
log_disconnections = 'on'
log_checkpoints = 'on'
log_lock_waits = 'on'
temp_tablespaces = 'ts_ydp10, ts_zcc31'
```

Этап 3. Дополнительные табличные пространства и наполнение базы

Создание директорий и табличных пространств

```
mkdir -p $HOME/ydp10
mkdir -p $HOME/zcc31
```

```
-- Табличные пространства для временных объектов
CREATE TABLESPACE ts_ydp10 LOCATION '/var/db/postgres0/ydp10';
CREATE TABLESPACE ts_zcc31 LOCATION '/var/db/postgres0/zcc31';

-- Назначить оба пространства для временных объектов
ALTER SYSTEM SET temp_tablespaces = 'ts_ydp10, ts_zcc31';
SELECT pg_reload_conf();
```

Создание базы данных

```
CREATE DATABASE bestbluemath TEMPLATE template1;
```

Создание роли и раздача прав

```
CREATE ROLE lab1_user WITH LOGIN PASSWORD 'lab1pass';

GRANT CONNECT ON DATABASE postgres      TO lab1_user;
GRANT CONNECT ON DATABASE bestbluemath  TO lab1_user;
GRANT CREATE  ON TABLESPACE ts_ydp10    TO lab1_user;
GRANT CREATE  ON TABLESPACE ts_zcc31    TO lab1_user;
GRANT CREATE  ON TABLESPACE pg_default  TO lab1_user;
GRANT USAGE  ON SCHEMA public TO lab1_user;
GRANT CREATE ON SCHEMA public TO lab1_user;
```

Отдельно выдаём права на схему `public` в базе `bestbluemath` (гранты на схему действуют в рамках конкретной базы):

```
psql -U postgres0 -d bestbluemath -p 9748 -c "GRANT USAGE ON SCHEMA public TO
lab1_user;"
psql -U postgres0 -d bestbluemath -p 9748 -c "GRANT CREATE ON SCHEMA public TO
lab1_user;"
```

Наполнение баз от имени новой роли

Скрипт выполняется от лица `lab1_user` через TCP/IP с паролем (MD5) в обе базы:

```
PGPASSWORD=lab1pass psql -h 127.0.0.1 -U lab1_user -d postgres     -p 9748 -f
seeds.sql
PGPASSWORD=lab1pass psql -h 127.0.0.1 -U lab1_user -d bestbluemath -p 9748 -f
seeds.sql
```

```
-- Таблица в табличном пространстве ts_ydp10
CREATE TABLE data_ydp10 (
    id         serial PRIMARY KEY,
    payload    text,
    created_at timestamp DEFAULT now()
) TABLESPACE ts_ydp10;

-- Таблица в табличном пространстве ts_zcc31
CREATE TABLE data_zcc31 (
    id         serial PRIMARY KEY,
    payload    text,
    created_at timestamp DEFAULT now()
) TABLESPACE ts_zcc31;

-- Таблица в пространстве по умолчанию
CREATE TABLE data_default (
    id         serial PRIMARY KEY,
    payload    text,
    created_at timestamp DEFAULT now()
);

-- Наполнение данными
INSERT INTO data_ydp10  (payload) SELECT 'record_' || g FROM generate_series(1,
200) g;
INSERT INTO data_zcc31  (payload) SELECT 'record_' || g FROM generate_series(1,
200) g;
INSERT INTO data_default(payload) SELECT 'record_' || g FROM generate_series(1,
200) g;

-- Временная таблица — использует temp_tablespaces (ts_ydp10 / ts_zcc31)
CREATE TEMP TABLE temp_data AS
    SELECT id, payload FROM data_default LIMIT 50;
```

Список табличных пространств и объектов

Для получения полного списка используется следующий запрос (выполняется от суперпользователя):

```sql
with ts_info as (
    select oid, spcname from pg_tablespace
),
db_ts as (
    select ts.spcname as ts_name, db.datname as obj_name,
           'database' as obj_type, NULL as schema_name
    from pg_database db join ts_info ts on db.dattablespace = ts.oid
),
rel_obj as (
    select coalesce(t.spcname, 'pg_default') as ts_name,
           c.relname as obj_name,
           case c.relkind
               when 'r' then 'table'     when 'i' then 'index'
               when 'S' then 'sequence'  when 'v' then 'view'
               when 'm' then 'materialized view'
               when 't' then 'TOAST table'
               else c.relkind::text
           end as obj_type,
           n.nspname as schema_name
    from pg_class c
    join pg_namespace n on c.relnamespace = n.oid
    left join ts_info t on c.reltablespace = t.oid
    where c.relkind in ('r','i','S','v','m','t')
)
select
    case when row_number() over (partition by ts_name
                                 order by obj_type, obj_name) = 1
         then ts_name else null
    end as "tablespace",
    case when schema_name is not null
         then schema_name || '.' || obj_name
         else obj_name
    end as "object",
    obj_type as "type"
from (select * from db_ts union all select * from rel_obj) all_obj
order by ts_name, obj_type, schema_name, obj_name;
```

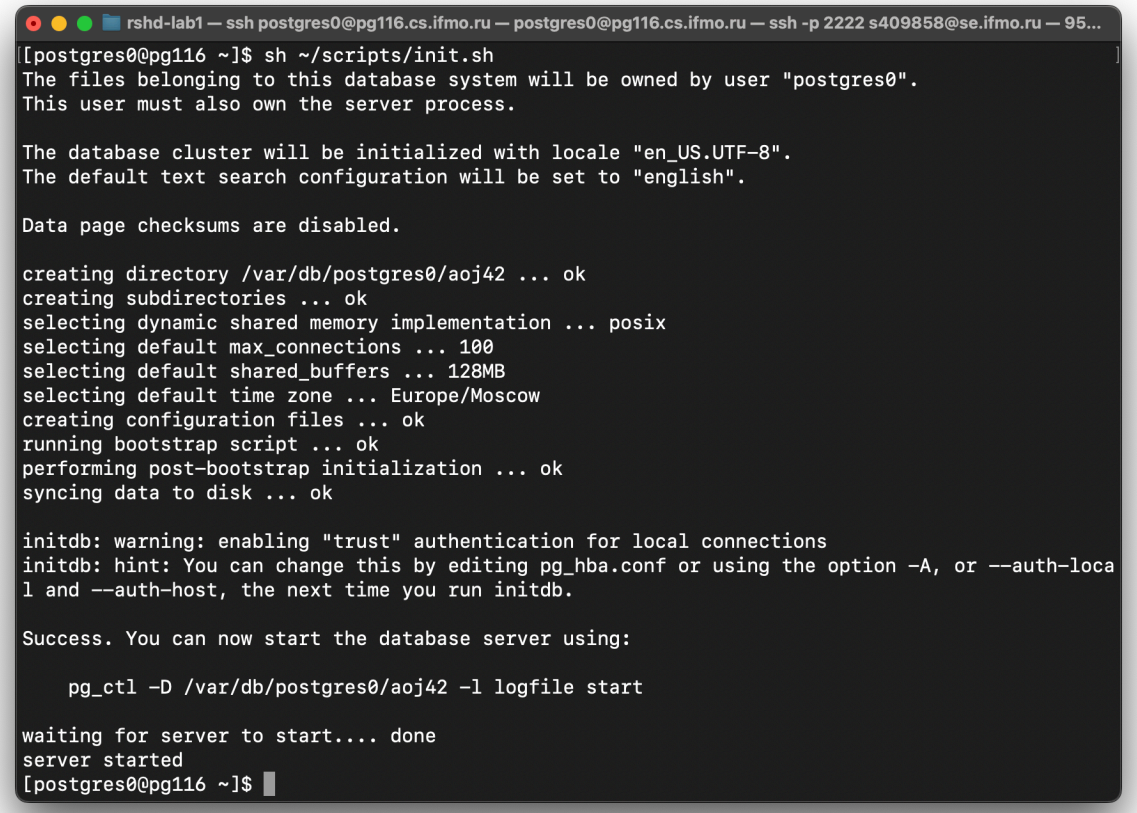
Фрагмент вывода (пользовательские объекты):

tablespace	object	type
pg_default	bestbluemath	database
	postgres	database
	template0	database
	template1	database
	public.data_default	table
	public.data_default_pkey	index
	public.data_default_id_seq	sequence
pg_global	pg_catalog.pg_authid	table

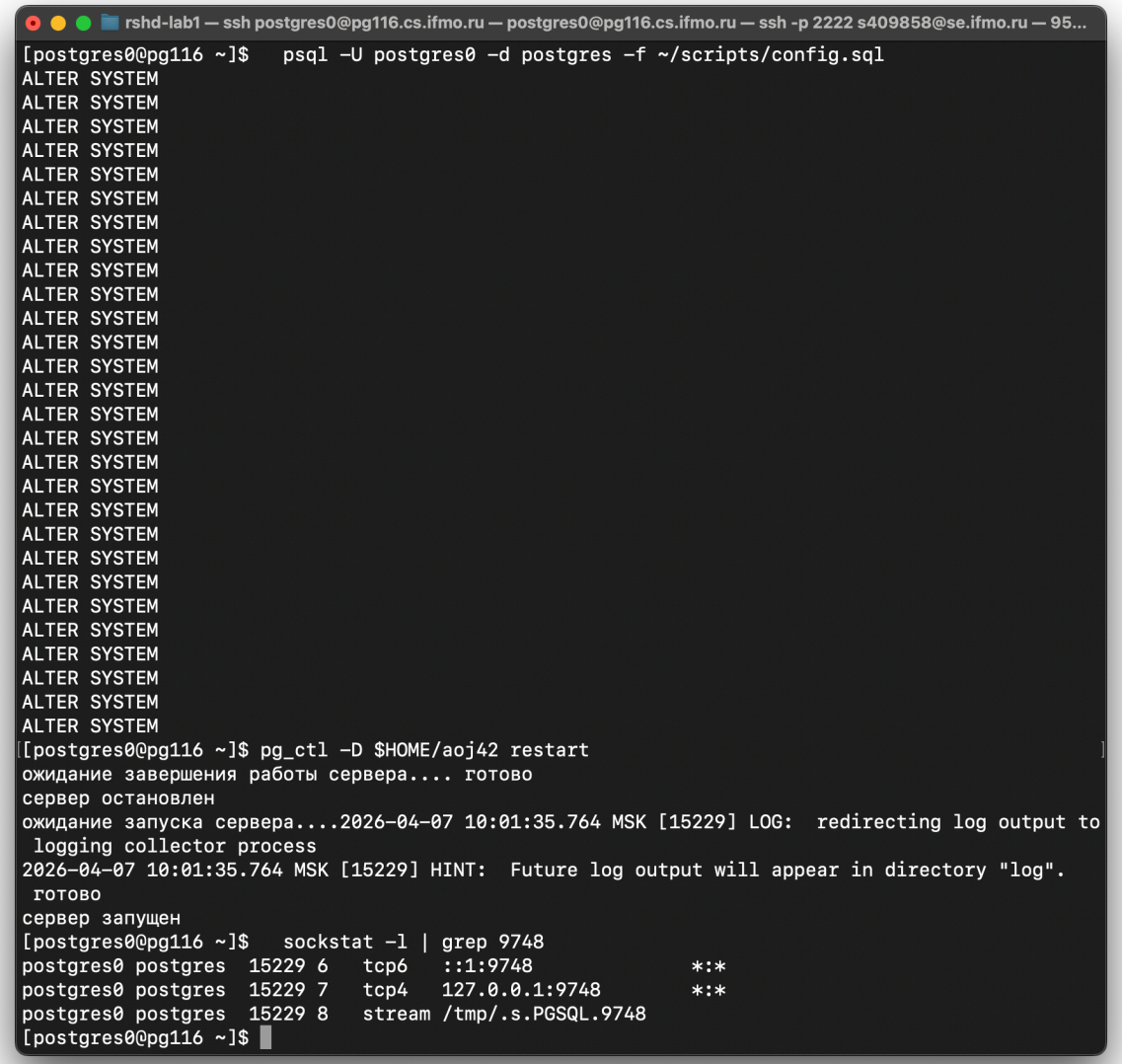
tablespace	object	type
	pg_catalog.pg_database	table
	...	...
ts_ydp10	public.data_ydp10	table
	pg_toast.pg_toast_16393	TOAST table
	pg_toast.pg_toast_16393_index	index
ts_zcc31	public.data_zcc31	table
	pg_toast.pg_toast_16403	TOAST table
	pg_toast.pg_toast_16403_index	index

Демонстрация

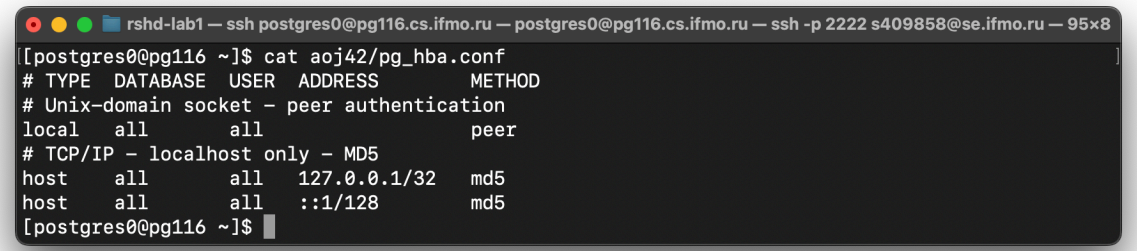
Инициализация кластера



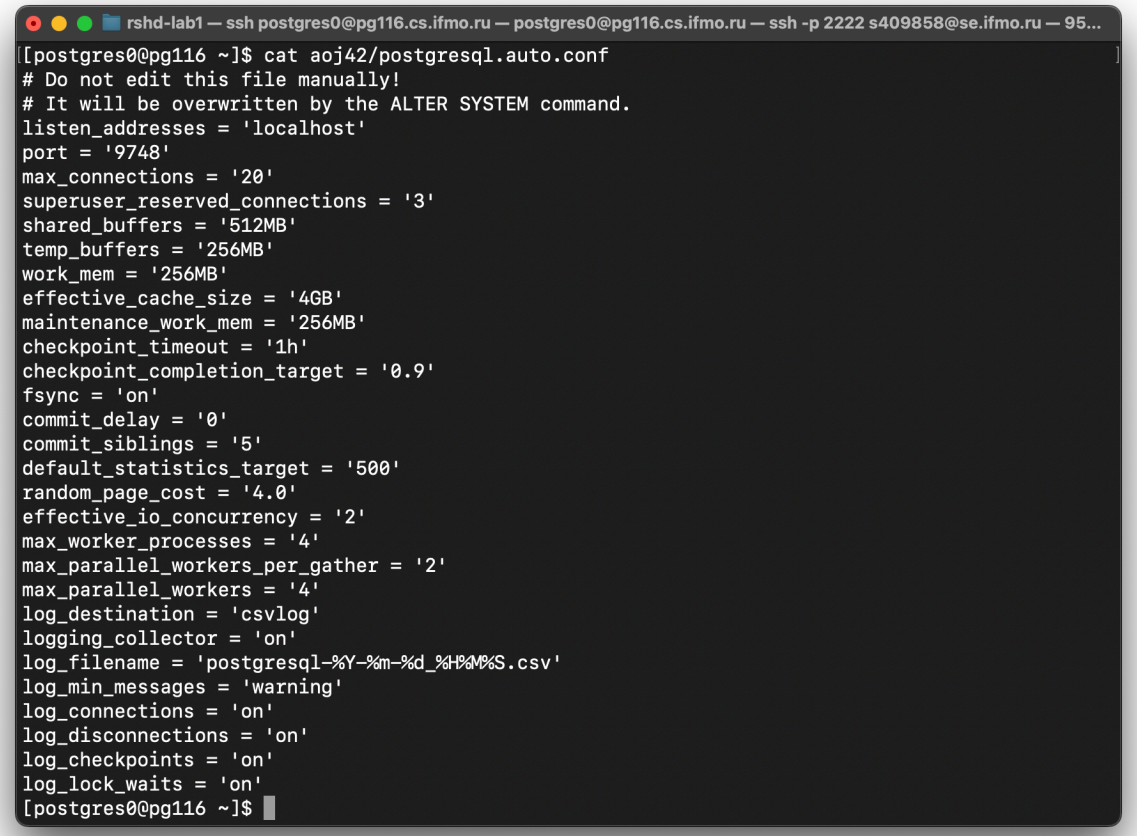
Запуск сервера и проверка порта



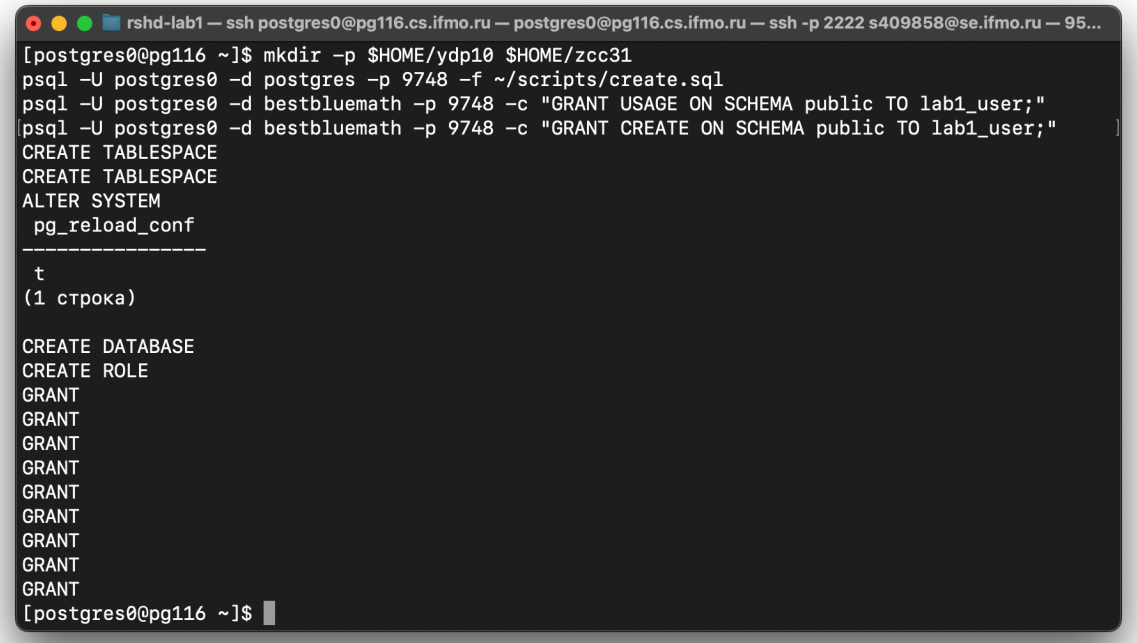
Содержимое pg_hba.conf



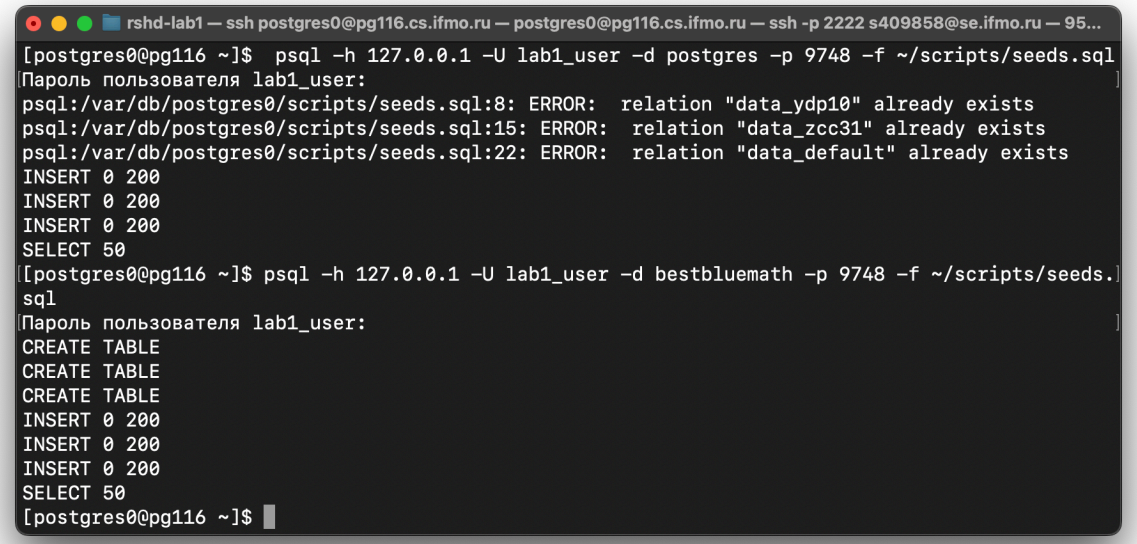
Применение конфигурации (postgresql.auto.conf)



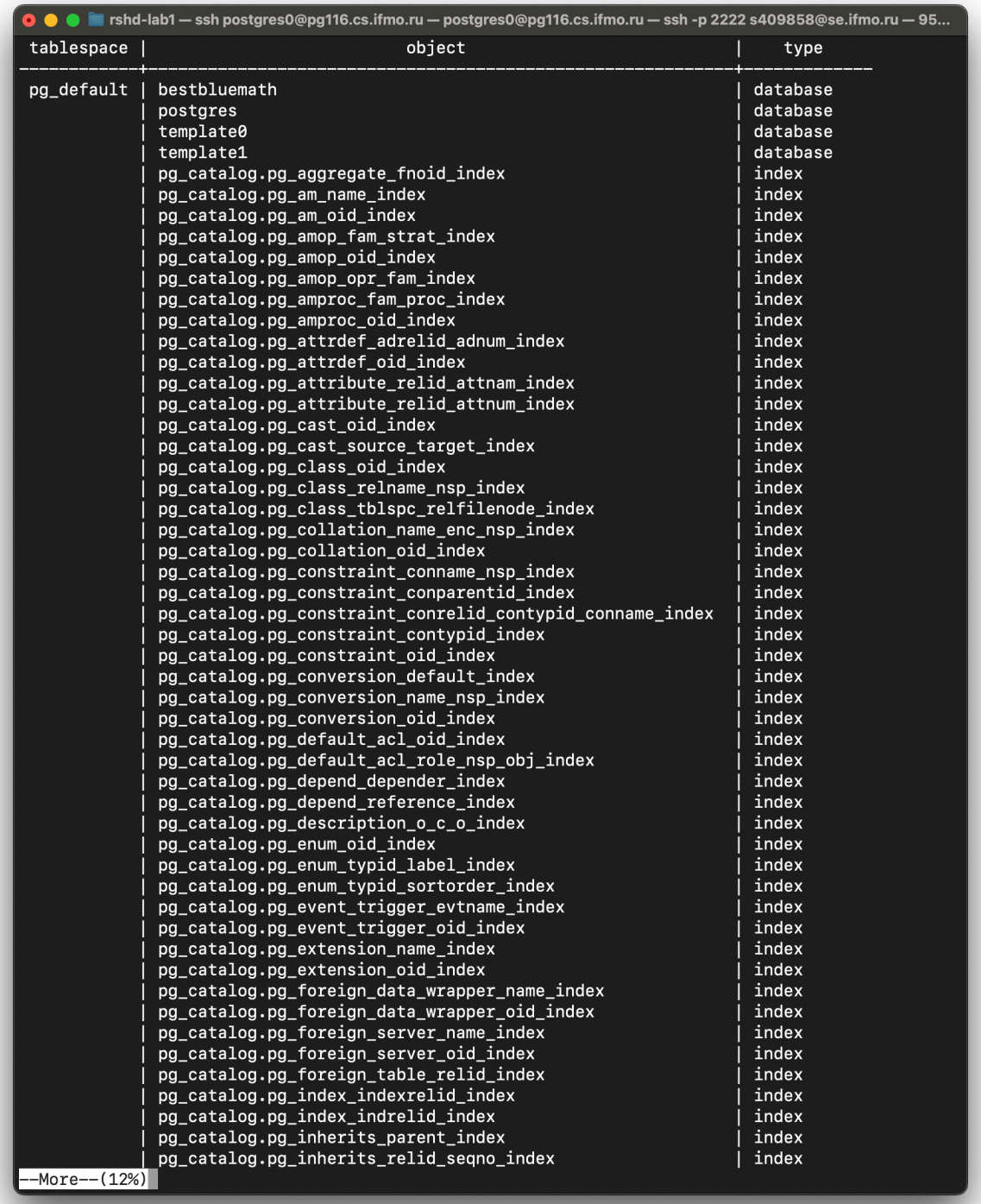
Создание табличных пространств и базы



Наполнение баз от имени lab1_user



Список табличных пространств и объектов



Исходный код

https://github.com/Vaneshik/rshd-lab1

Скрипты лабораторной работы:

- `scripts/init.sh` — инициализация кластера и запуск сервера
- `scripts/config.sql` — конфигурация параметров сервера через `ALTER SYSTEM`
- `scripts/create.sql` — создание табличных пространств, базы данных и роли
- `scripts/seeds.sql` — наполнение баз тестовыми данными
- `scripts/table_spaces.sql` — вывод всех табличных пространств и объектов
- `scripts/clean.sh` — очистка кластера и директорий

Вывод

В ходе выполнения лабораторной работы был успешно создан и настроен кластер PostgreSQL с использованием выделенного узла.